University of Ha'il

SENG 361 / SENG 362: Web Engineering

Course Assignment: CampusEats SaaS Application

Student Information:

- **Name:** rakan alhrbi
- **Student ID:** 202207662
- **Course Section** SENG 362 (Web Engineering)
- **University:** University of Ha'il

Part A — Requirements with BDD

1. User Stories

- **For the Student:**

As a university student,

I want to add food items to my cart dynamically without reloading the page,

So that I can complete my meal order quickly and efficiently between lectures.

SMART Criteria Check:

  - **Specific:** Focuses directly on adding food items dynamically to the cart.
  - **Measurable:** Success is verified when cart total and items update on screen with zero page reloads.
  - **Achievable:** Implemented using standard Rails controllers and AJAX (Fetch API).
  - **Relevant:** Solves the core user pain point of slow food court ordering.
  - **Time-bound:** Scheduled for delivery within Sprint 1.
- **For the Vendor:**

As a food court vendor,

I want to update menu item availability in real time (e.g., mark as sold out),

So that students cannot order items that are out of stock.

SMART Criteria Check:

  - **Specific:** Controls item availability status in the database.
  - **Measurable:** Verified when sold-out items prevent cart submission and return appropriate error messages.
  - **Achievable:** Handled via database boolean flags and model validations in Rails.
  - **Relevant:** Protects vendor operational reliability and customer satisfaction.
  - **Time-bound:** Scheduled for delivery within Sprint 1.
- **For the Food-Court Manager:**

As a food-court manager,

I want to view a consolidated overview of all vendors and their structured menu items,

So that I can oversee campus dining operations effectively.

SMART Criteria Check:

- **Specific:** Displays nested vendor and menu relationships.
- **Measurable:** Verified by accessing an admin index view listing all active vendors and items.
- **Achievable:** Implemented using Active Record association queries.
- **Relevant:** Serves essential administrative and reporting requirements.
- **Time-bound:** Scheduled for delivery within Sprint 2.

2. Cucumber Feature File (`campus_eats.feature`)

مقتطف الرمز

```
Feature: CampusEats Ordering
  As a student
  I want to place food orders from the campus vendors
  So that I can get my meals efficiently

  Scenario: Successful order placement
    Given the user is on the menu page for "Burger House"
    When the user adds "Chicken Burger" to the cart
    Then the cart total should update to "SAR 25.00"
    And the order should be saved successfully in the system

  Scenario: Ordering an item that is sold out
    Given the user is on the menu page for "Burger House"
    And the item "Crispy Fries" is marked as sold out
    When the user attempts to add "Crispy Fries" to the cart
    Then an error message should display stating "Item is sold out"
    And the cart item count should remain unchanged
```

Why Declarative Steps Are Better Than Imperative Steps:

Declarative BDD steps describe **what** business behavior is happening rather than **how** the technical UI low-level interaction occurs (such as clicking specific HTML buttons, waiting N seconds, or filling form inputs). Declarative feature files remain clear and readable for non-technical stakeholders, and they are significantly less fragile when user interface layouts change over time.

3. Sketches, Estimation, and Calculations

Lo-fi UI Design Description:

1. **Menu Page Screen:**
    a. **Header Section:** Displays the selected vendor's name and location.
    b. **Main Grid:** Cards for each menu item showing image, title, price, availability tag, and an accessible `<button>` labeled "Add to Cart".
    c. **Cart Sidebar:** Displays selected items, dynamic quantities, live total sum, and a "Checkout" button.
2. **Cart Screen:**
    a. **Tabular View:** Displays Item Name, Unit Price, Quantity Selector, Line Total, and Action buttons (Remove/Update).
    b. **Summary Box:** Subtotal, VAT/Taxes, Grand Total (SAR), and Order Status Indicator.

Estimation & Cost Calculations:

- **Total Story Points:** Estimated at **24 points** across all primary user stories (Fibonacci scale: 5 points for Student story, 8 points for Vendor story, 3 points for Manager story, plus 8 points for cart/checkout integration).
- **Team Velocity:** 12 points per 2-week sprint.
- **Cost per Point:** SAR 1,200.

Mathematical Calculations:

1. **Number of Sprints:**

$$\text{Sprints} = \frac{\text{Total Story Points}}{\text{Velocity}} = \frac{24 \text{ points}}{12 \text{ points/sprint}} = 2 \text{ Sprints (4 Weeks)}$$

2. **Total Financial Cost:**

$$\text{Total Cost} = \text{Total Story Points} \times \text{Cost per Point} = 24 \times 1,200 = 28,800 \text{ SAR}$$

Part B — Rails Back End

1. Migrations & Models Relationships

Execute the following terminal commands to generate migrations and models:

Bash
```
rails g model Vendor name:string location:string
rails g model MenuItem name:string price:decimal description:text
available:boolean vendor:references
rails g model Order user_name:string total_price:decimal
status:string
rails g model OrderItem order:references menu_item:references
quantity:integer
rails db:migrate
```

- **Models Source Code (`app/models/`):**

`vendor.rb`

Ruby
```
class Vendor < ApplicationRecord
  has_many :menu_items, dependent: :destroy
end
```

`menu_item.rb`

Ruby
```
class MenuItem < ApplicationRecord
  belongs_to :vendor
  has_many :order_items
  has_many :orders, through: :order_items

  validates :name, presence: true
  validates :price, presence: true, numericality: { greater_than: 0
}
end
```

`order.rb`

Ruby
```
class Order < ApplicationRecord
  has_many :order_items, dependent: :destroy
```

```
  has_many :menu_items, through: :order_items
end
```

order_item.rb

Ruby
```
class OrderItem < ApplicationRecord
  belongs_to :order
  belongs_to :menu_item

  validates :quantity, numericality: { only_integer: true,
greater_than_or_equal_to: 1, less_than_or_equal_to: 10 }
  validate :item_must_be_available

  private

  def item_must_be_available
    if menu_item && !menu_item.available
      errors.add(:base, "Selected item is sold out and cannot be
added to the order.")
    end
  end
end
```

2. Routes & The 6 Routes Table (`config/routes.rb`)

Ruby
```
Rails.application.routes.draw do
  resources :vendors, only: [:index, :show] do
    resources :menu_items, only: [:index, :show] do
      resources :order_items, only: [:create, :destroy]
    end
  end
  resources :orders, only: [:show]
end
```

HTTP Method	URI Pattern	Controller#Action	Description

GET	`/vendors/:vendor_id/menu_items`	`menu_items#index`	List menu items for a specific vendor
GET	`/vendors/:vendor_id/menu_items/:id`	`menu_items#show`	Show specific menu item details
POST	`/vendors/:vendor_id/menu_items/:menu_item_id/order_items`	`order_items#create`	Add item to cart via AJAX
DELETE	`/vendors/:vendor_id/menu_items/:menu_item_id/order_items/:id`	`order_items#destroy`	Remove item from cart
GET	`/orders/:id`	`orders#show`	Show order cart and summary

3. OrderItemsController (app/controllers/order_items_controller.rb)

Ruby

```
class OrderItemsController < ApplicationController
  before_action :set_order, only: [:create, :destroy]

  def create
    @menu_item = MenuItem.find(params[:menu_item_id])
    @order_item = @order.order_items.new(menu_item: @menu_item,
quantity: params[:quantity] || 1)

    respond_to do |format|
```

```
      if @order_item.save
        format.json { render json: { status: 'success', cart_total:
calculate_cart_total, html: render_to_string(partial: 'order_item',
locals: { order_item: @order_item }, formats: [:html]) }, status:
:created }
      else
        format.json { render json: { status: 'error', message:
@order_item.errors.full_messages.join(', ') }, status:
:unprocessable_entity }
      end
    end
  end

  def destroy
    @order_item = @order.order_items.find(params[:id])
    @order_item.destroy

    respond_to do |format|
      format.json { render json: { status: 'success', cart_total:
calculate_cart_total } }
    end
  end

  private

  def set_order
    @order = Order.find_or_create_by(id: session[:order_id]) do |o|
      session[:order_id] = o.id
    end
  end

  def calculate_cart_total
    @order.order_items.sum("quantity * (SELECT price FROM menu_items
WHERE menu_items.id = order_items.menu_item_id)")
  end
end
```

Partial Template (app/views/order_items/_order_item.html.erb):

مقتطف الرمز

```
<tr id="order_item_<%= order_item.id %>">
  <td><%= order_item.menu_item.name %></td>
  <td><%= order_item.quantity %></td>
```

```
  <td>SAR <%= order_item.menu_item.price * order_item.quantity
%></td>
  <td><%= link_to 'Remove',
vendor_menu_item_order_item_path(order_item.menu_item.vendor,
order_item.menu_item, order_item), method: :delete, remote: true,
class: 'text-danger' %></td>
</tr>
```

Part C — JavaScript Client

JavaScript

```
document.addEventListener("DOMContentLoaded", () => {
  const addButtons = document.querySelectorAll(".add-to-cart-btn");

  addButtons.forEach(button => {
    button.addEventListener("click", async (e) => {
      const vendorId = button.dataset.vendorId;
      const menuItemId = button.dataset.menuItemId;

      button.disabled = true;
      const originalText = button.innerText;
      button.innerText = "Adding...";
      button.setAttribute("aria-busy", "true");

      try {
        const response = await
fetch(`/vendors/${vendorId}/menu_items/${menuItemId}/order_items`, {
          method: "POST",
          headers: {
            "Content-Type": "application/json",
            "X-CSRF-Token": document.querySelector('meta[name="csrf-
token"]').content,
            "Accept": "application/json"
          },
          body: JSON.stringify({ quantity: 1 })
        });

        const data = await response.json();

        if (response.ok && data.status === 'success') {
          document.querySelector("#cart-items-
table").insertAdjacentHTML('beforeend', data.html);
```

```
          document.querySelector("#cart-total").innerText = `SAR
${data.cart_total}`;
        } else {
          alert(`Error: ${data.message || "Item is sold out or
unavailable."}`);
        }
      } catch (error) {
        alert("Network error occurred. Please check your
connection.");
      } finally {
        button.disabled = false;
        button.innerText = originalText;
        button.setAttribute("aria-busy", "false");
      }
    });
  });
});
```

Part D — Reflection

1. **Rails Pitfall (N+1 Query Problem):**

The N+1 query problem can severely slow down the cart or order listing page when rendering each order item alongside its associated menu item details individually. To avoid this performance bottleneck, we use Active Record's `.includes(:menu_item)` method (e.g., `order.order_items.includes(:menu_item)`) to eager-load the necessary associations in a single optimized SQL query, preventing database overload.

2. **Network Fallacy in AJAX Code:**

When designing our AJAX/Fetch client-side code, we had to handle network fallacies such as latency, unreliable connections, or server timeouts. We addressed this by implementing a `try...catch` block around the `fetch` request, displaying an explicit alert or error message to the user if the network fails, and disabling the submit button with a loading indicator to prevent multiple duplicate requests while waiting for a response.